Info:
This code changes the output color of several LED lights based on user input from a button and potentiometer. After reading the button and potentiometer input, the potentiometer input is mapped to a hue value between 0 and 330, in order to account for the receiver. Then, the hue is converted to an RGB color with a saturation and value of 1. After that, if the button is not pressed, the output color is overwritten with #000000, resulting in the LEDs being turned off. Finally, each LED in each chain is set to the output color.

```
#include <FastLED.h>

// Requires FastLED library

// Constants
const int LED_COUNT = 3;              // LED's per chain
const int LED_ROW_COUNT = 1;          // number of pins required for
however many LED chains being used
const double TWO = 2;                 // modf() requires a pointer for its
second parameter for some reason
const int INPUT_MAX_VALUE = 1023;     // max value of potentiometer input
const int INPUT_PIN = A4;             // potentiometer pin
const int BUTTON_PIN = 3;             // button pin
const int MAX_HUE = 330;              // maximum output hue
// LED_PIN has to be hardcoded, edit that in setup() { }

// Instance Variables
int rgbOut[3] = { 0, 0, 0 };          // output from HueToRGB() -- { 0 ==
r, 1 == g, 2 == b }
int hue = 0;                          // the hue
int input = 0;                        // the potentiometer input
bool lightOn = false;                 // the button input
CRGB leds[LED_COUNT];                 // the LEDs

void setup() {
  Serial.begin(9600);
  delay(500);

  // set pinModes
```

```
  FastLED.addLeds<WS2812, 2, GRB>(leds, LED_COUNT);   // 2 == pin 2, must
be hardcoded here
  pinMode(INPUT_PIN, INPUT);
  pinMode(BUTTON_PIN, INPUT);
}

void loop() {
  // get inputs
  input = analogRead(INPUT_PIN);
  lightOn = (digitalRead(BUTTON_PIN) == HIGH);

  // get the hue to use here
  hue = map(input, 0, INPUT_MAX_VALUE, 0, MAX_HUE) % 360;

  // setup rgbOut[]
  HueToRGB(hue);

  // if should be off
  if(!lightOn) {
    rgbOut[0] = 0;
    rgbOut[1] = 0;
    rgbOut[2] = 0;
  }

  // set LED colors
  for(int j = 0; j < LED_COUNT; j++) {
    leds[j] = CRGB(rgbOut[0], rgbOut[1], rgbOut[2]);
  }
  FastLED.show();

  // debug output
  // ex: "hue(0) = [255, 0, 0]"
  Serial.print("hue(");
  Serial.print(hue);
  Serial.print(") = [");
  Serial.print(rgbOut[0]);
  Serial.print(", ");
  Serial.print(rgbOut[1]);
  Serial.print(", ");
  Serial.print(rgbOut[2]);
```

```
  Serial.println("]");
  // Serial.println(input);

  // temporary - shows the color can change
  // hue += 1;
  // hue = hue % 360;
  // delay(33);
}

void HueToRGB(int h) {
  // initialize to #000000
  rgbOut[0] = 0;
  rgbOut[1] = 0;
  rgbOut[2] = 0;

  // hueOffset = variable hue offset
  int hueOffset = (int)(255 * (1 - abs(modf((h / 60.0), &TWO) - 1)));

  // sector = the sector of a circular representation of the hue that h is
in
  int sector = h / 60;

  // set the color based on sector and hueOffset
  if(sector == 0) {
    rgbOut[0] = 255;
    rgbOut[1] = hueOffset;
  } else if(sector == 1) {
    rgbOut[0] = 255 - hueOffset;
    rgbOut[1] = 255;
  } else if(sector == 2) {
    rgbOut[1] = 255;
    rgbOut[2] = hueOffset;
  } else if(sector == 3) {
    rgbOut[1] = 255 - hueOffset;
    rgbOut[2] = 255;
  } else if(sector == 4) {
    rgbOut[2] = 255;
    rgbOut[0] = hueOffset;
  } else if(sector == 5) {
    rgbOut[2] = 255 - hueOffset;
```

```
    rgbOut[0] = 255;
  }
}
```